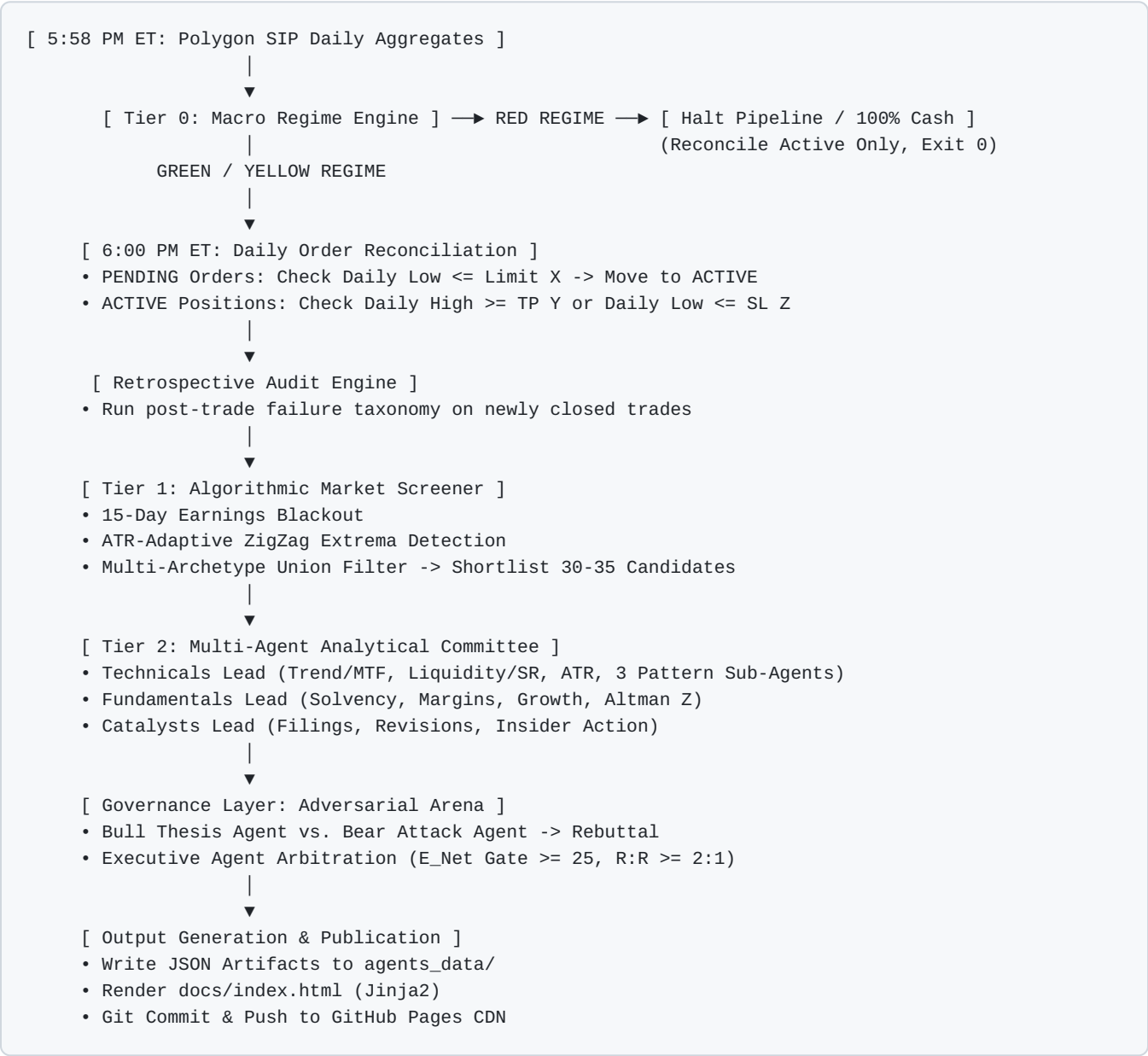


Argus Trader: Phase 1 Product Design Document

System Overview: Argus Trader is an automated, multi-agent swing trading architecture for liquid U.S. equities (~1,000 tickers). It couples deterministic market-structure pipelines with an adversarial LLM decision committee, bound by quantitative risk constraints, an order state machine, a static GitHub Pages operational interface, and automated CI/CD backtesting regression gates.

1. System Architecture & Daily Batch Pipeline

The daily production cycle runs deterministically post-market close between **5:58 PM ET** and **6:15 PM ET**.



2. Data Layer & Historical Ingestion

Data Source

- **Provider:** Polygon.io Consolidated Tape (SIP) via Developer/Starter tier.
- **Endpoint:** Grouped Daily Bars (`/v2/aggs/grouped/locale/us/market/stocks/{date}`).
- **Window:** 5-year rolling history covering ~1,000 liquid equities (>\$10 share price, >1M average daily volume), plus index benchmarks (SPY, QQQ, VIX).

Partitioning & GitHub Storage Strategy

To comply with GitHub's 50 MB warning and 100 MB hard limit per file, raw ingestion data is compressed with Snappy and partitioned quarterly:

- **Format:** Columnar Apache Parquet (`.parquet`).
- **Path Schema:** `data/raw/polygon/{YYYY}/{YYYY}_Q{1-4}.parquet`
- **Quarter Volume:** ~63 trading sessions per quarter × ~1,000 tickers ≈ 63,000 rows.
- **Storage Footprint:** ~8 MB to 12 MB per partition file (safely below GitHub limits).

3. Tier 0: Macro Regime Circuit Breaker

Evaluates broad market health at 5:58 PM ET before executing scans or allocating capital.

Parameter	Green Regime (Risk-On)	Yellow Regime (Caution / Distribution)	Red Regime (Hostile / Crash)
SPY Moving Averages	Close > EMA ₅₀ AND > SMA ₂₀₀	Close < EMA ₅₀ BUT > SMA ₂₀₀	2 consecutive daily closes < SMA ₂₀₀
QQQ Trend	Close > SMA ₂₀₀	Close within 2% of SMA ₂₀₀	Close < SMA ₂₀₀
VIX Volatility	VIX < 21.0	21.0 ≤ VIX ≤ 28.5	VIX > 28.5
Breadth (%S ₅₀)	≥ 50% above 50 EMA	30% ≤ %S ₅₀ < 50%	< 30% above 50 EMA
Risk Per Trade	1.0% total equity	0.5% total equity	0.0% (Trading Halted)
Max Open Trades	6 to 8 positions	3 positions	0 positions (100% Cash)
Eligible Setups	All Archetypes (A, B, C)	Archetypes A & B only	Pipeline skips Tiers 1 & 2

4. Tier 1: Algorithmic Market Screener

Screens ~1,000 tickers down to a shortlist of 30–35 actionable setups using deterministic Python logic.

1. Mandatory Invalidation Gates

- **Liquidity:** 20-day Average Daily Volume $\geq 1,000,000$ shares; Close $\geq \$10.00$.
- **Earnings Blackout:** Hard drop if corporate earnings occur within the next 15 calendar days.

2. Extrema Detection Engine: ATR-Adaptive ZigZag

Swing pivots are calculated via volatility-adjusted price reversals rather than fixed-width time windows:

$$\text{Reversal Threshold} = 1.5 \times \text{ATR}_{14}$$

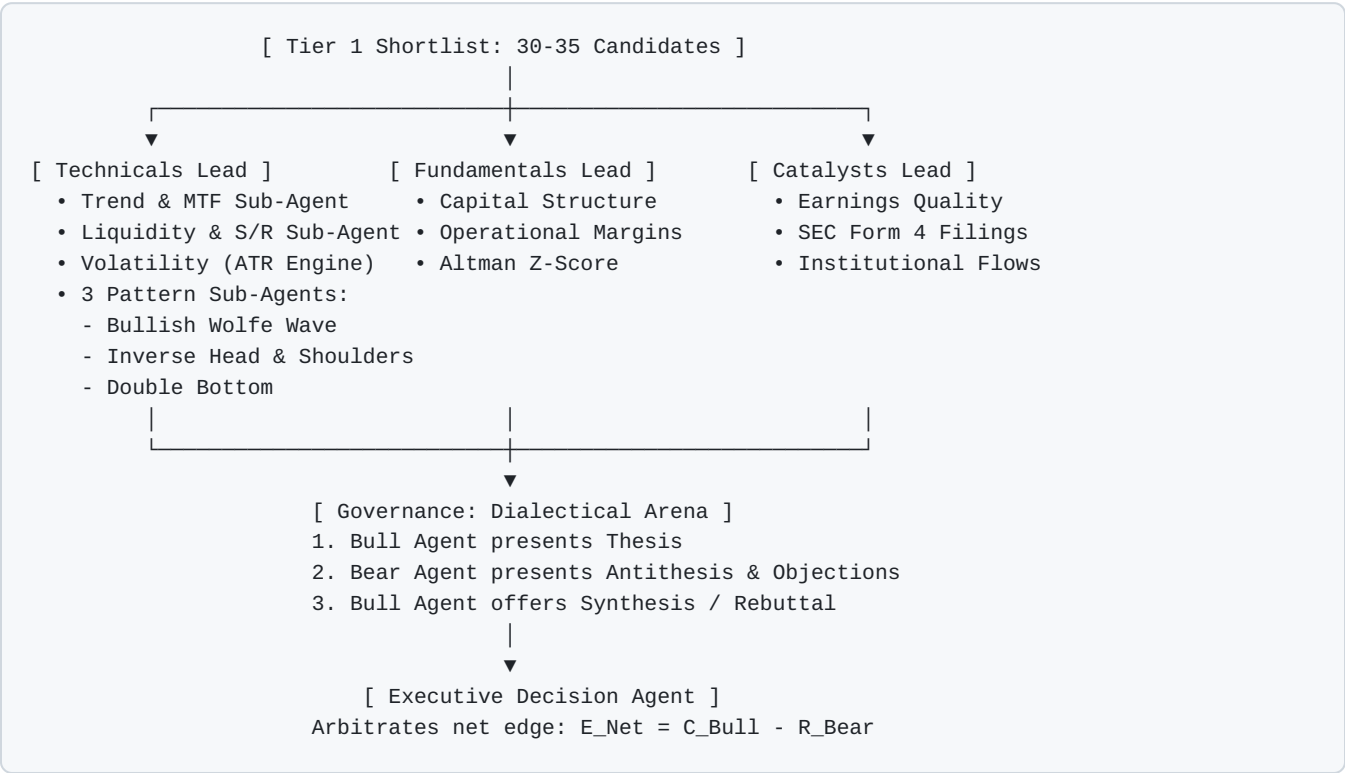
- Guarantees alternating Peak $\rightarrow$ Trough $\rightarrow$ Peak sequence.
- Extracts exact (Date, Price) coordinate pairs to feed Tier 2 geometric pattern agents.

3. Setup Archetypes & Scoring

- **Archetype A (Trend Pullback):** Daily Close $> \text{SMA}_{200}$; $20 \leq \text{RSI}_{14} \leq 45$; price touching or within $1.0 \times \text{ATR}_{14}$ of rising 50-day EMA.
- **Archetype B (Volatility Squeeze):** Bollinger Bands (20, 2.0) fully contract inside Keltner Channels (20, 1.5); volume contracting ≥ 3 consecutive sessions.
- **Archetype C (Relative Strength / Momentum Consolidation):** 3-month relative performance $> \text{SPY}$; consolidation within 5% of 52-week high; base duration ≥ 10 sessions.

5. Tier 2: Multi-Agent Committee & Adversarial Debate

Shortlisted candidates are analyzed across three independent analytical domains before entering an adversarial debate.



Analytical Domains

1. Technicals Domain:

- **Trend & MTF Sub-Agent:** Verifies weekly/daily moving average alignment and Stage 2 markup confirmation.
- **Liquidity & S/R Sub-Agent:** Maps Volume Profile Point of Control (POC), Value Area High (VAH), and Value Area Low (VAL).
- **Volatility Engine (Pure Python):** Calculates ATR buffers and invalidation distances.
- **Pattern Sub-Agents (Wolfe Wave, Inverse H&S, Double Bottom):** Evaluates geometric symmetry, pivot alignments, and neckline breakout volumes.

2. Fundamentals Domain:

Audits solvency, debt/equity ratios, operating cash flow, and flags distress risk via the Altman Z-score ($Z < 1.8$ triggers an automatic veto).

3. Catalysts Domain:

Audits upcoming macro catalysts, earnings revisions, and Form 4 insider transactions.

Adversarial Arbitration Protocol

- **Bull Agent:** Formulates optimal entry limit (X), structural target (Y), and base thesis score ($C_{Bull} \in [0, 100]$).
- **Bear Agent:** Attacks structural flaws, overhead resistance, and sector headwinds, returning an objection risk score ($R_{Bear} \in [0, 100]$).
- **Executive Decision Agent:** Approves trade tickets only when:

$$E_{Net} = C_{Bull} - R_{Bear} \geq 25 \text{ where } C_{Bull} \geq 75 \text{ and } R_{Bear} < 50$$

- **Reward-to-Risk Hurdle:** Minimum 2.0:1 calculated strictly against structural invalidation points.

Agent Artifact Hierarchy

All agent evaluations and debate transcripts are serialized to disk:

```
artifacts/agents_data/
├── technicals/
│   ├── parent_lead/{YYYY-MM-DD}/{TICKER}_technicals_dossier.json
│   ├── trend_mtf_sub_agent/{YYYY-MM-DD}/...
│   ├── liquidity_sr_sub_agent/{YYYY-MM-DD}/...
│   ├── wolfe_wave_sub_agent/{YYYY-MM-DD}/...
│   ├── inverse_hs_sub_agent/{YYYY-MM-DD}/...
│   └── double_bottom_sub_agent/{YYYY-MM-DD}/...
├── fundamentals/
│   └── fundamental_lead/{YYYY-MM-DD}/{TICKER}_fundamentals_dossier.json
├── catalysts/
│   └── catalyst_lead/{YYYY-MM-DD}/{TICKER}_catalysts_dossier.json
└── governance/
    ├── bull_bear_debates/{YYYY-MM-DD}/{TICKER}_debate.json
    └── executive_orders/{YYYY-MM-DD}/approved_tickets.json
```

6. Portfolio Risk, Sizing & Capital Allocation

- **Account Risk Allocation:**

- Green Regime: 1.0% total equity risk per trade.
- Yellow Regime: 0.5% total equity risk per trade.
- Red Regime: 0.0% (all new orders blocked).

- **Position Sizing Formulas:**

Total Dollar Risk = Account Equity × Risk Percentage

Risk Per Share = Entry Limit (X) - Stop Loss (Z)

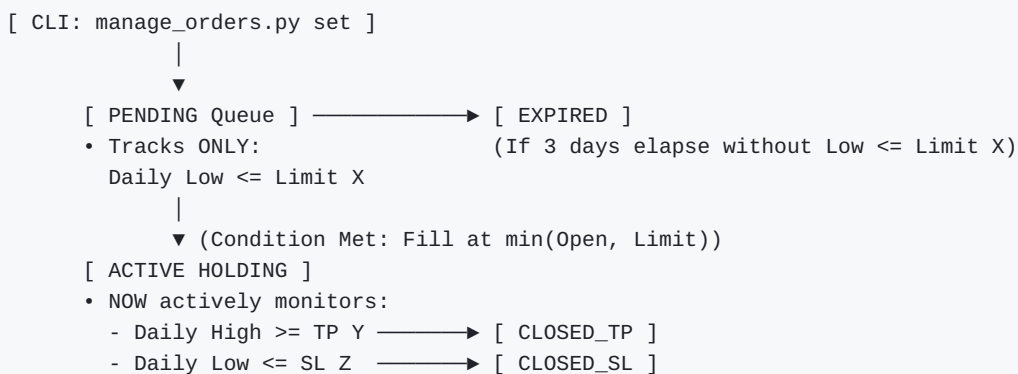
Shares to Buy = ⌊ Total Dollar Risk / Risk Per Share ⌋

- **Concentration Boundaries:**

- **Max Position Cap:** No single position may exceed 20% of total account equity, regardless of how tight the stop loss is.
- **Sector Exposure Cap:** Maximum of 25% of total portfolio equity in any single GICS sector.
- **Sub-Industry Cap:** Maximum of 2 concurrent open positions within the same industry sub-sector.
- **Leverage:** Maximum gross exposure ≤ 100% (zero margin/leverage).

7. Execution Architecture & Order State Machine

Argus Trader uses a local CLI utility (`manage_orders.py`) to stage orders, while the evening batch engine deterministically tracks order state transitions.



State Machine Lifecycle Rules

1. PENDING State:

- Holds orders staged via CLI: Limit Price (**X**), Take Profit (**Y**), Stop Loss (**Z**), and Share Quantity (**N**).
- Evaluates only one condition: `Daily Low ≤ Limit Price`.
- **Take Profit and Stop Loss tracking are completely dormant while PENDING.**
- If unfilled after 3 market sessions, transitions to `EXPIRED`.

2. ACTIVE State:

- Entered automatically when a pending order is filled.
- Begins monitoring exits on subsequent sessions:
 - **Take Profit Exit:** `Daily High ≥ TP Y` → exits at `max(Open, Y)`; logs `CLOSED_TP`.
 - **Stop Loss Exit:** `Daily Low ≤ SL Z` → exits at `min(Open, Z)`; logs `CLOSED_SL`.

3. Execution Anomaly Tracking (Empirical Gap & Collision Metrics):

- *Same-Bar Collisions:* If a single bar has both `High ≥ Y` and `Low ≤ Z`, the conservative backtesting rule executes the Stop Loss first. The event is incremented in `same_bar_bracket_collisions`.
- *Down-Gap Slippage:* If a stock gaps down below the Stop Loss (`Open < Z`), execution occurs at the Open price. Realized slippage is logged in `slippage_drag_dollars`.

CLI Interface Specification

```
# Stage a new pending limit order
python scripts/manage_orders.py set NVDA --limit 128.50 --tp 142.50 --sl 122.10 --shares 150

# Inspect active pipeline status
python scripts/manage_orders.py status
```

8. Dashboard & Public Operations Log (GitHub Pages)

The user interface is a static HTML/CSS single-page dashboard compiled via Jinja2 templates (`scripts/publish_dashboard.py`) and hosted directly on GitHub Pages (`docs/index.html`).

- **Macro Regime Banner:** Displays current market state (GREEN, YELLOW, RED), benchmark trends (SPY, QQQ, VIX), and breadth percentage.
- **Actionable Orders Table (Tomorrow's Session):** Displays valid `PENDING` orders with target entry, dynamic R:R, and remaining time-to-live (`Day N of 3`).
- **Active Portfolio Holdings:** Displays currently filled positions, unrealized P&L, days held, and distance to target/stop.
- **Historical Audit Ledger & Scorecard:** Displays completed trades, realized `R`-multiples, win rate, profit factor, and maximum drawdown.
- **Privacy Controls:** Shows risk allocation in **risk units (`R`)** and **portfolio percentages** to allow public dashboard hosting without broadcasting raw personal dollar balances.

9. Retrospective Audit Engine

Executes immediately following daily order reconciliation whenever a trade transitions to `CLOSED_TP`, `CLOSED_SL`, or `EXPIRED`.

Root-Cause Taxonomy

- **THESIS_VALIDATED (Win)**: Hit Take Profit within 15 sessions; volume confirmed the measured move.
- **FALSE_BREAKOUT (Loss)**: Stopped out within 3 sessions of entry; price failed to hold breakout level.
- **OVERHEAD_ABSORPTION (Loss)**: Stalled and reversed at overhead supply (POC/200 SMA) flagged during the Bear debate.
- **VOLATILITY_WHIPSAW (Loss)**: Low pierced Stop Loss by $< 0.25 \times \text{ATR}$ before reversing directly toward the target.
- **TIME_STOP_DECAY (Scratch/Loss)**: Churns sideways for > 15 sessions without hitting target.
- **MACRO_DRAG (Loss)**: Stopped out during an intraday market-wide liquidity drop or Tier 0 regime downgrade.

The engine writes audit notes to `data/audit_ledger.json` and outputs parameter-tuning recommendations to calibrate sub-agent scoring weights.

10. Testing, Verification & CI/CD Regression Gates

Every pull request against `main` must pass an automated 5-year simulation run. Merges are blocked if new logic degrades strategy performance.

Test Results Directory Structure

```
test_results/
├── benchmark_latest.json                # Production performance baseline
├── {YYYY-MM-DD}_PR-{ID}_{COMMIT_SHA}/
│   ├── benchmark_summary.json          # Candidate run metrics
│   ├── closed_trades.parquet           # Trade-by-trade ledger
│   ├── retrospective_audit.md          # Analysis of failures & recommendations
│   └── anomaly_report.json             # Same-bar collisions and gap counts
```

Pull Request Regression Gate Rules

The script `scripts/verify_regression_gate.py` compares the candidate `benchmark_summary.json` against `benchmark_latest.json`. A PR is blocked if any of the following occur:

1. **Take Profit Hit Rate Regression**: $TP\ Rate_{Candidate} < TP\ Rate_{Baseline}$
 2. **Expectancy Regression**: $Expectancy\ (R)_{Candidate} < Expectancy\ (R)_{Baseline}$
 3. **Profit Factor Degradation**: $Profit\ Factor_{Candidate} < Profit\ Factor_{Baseline}$
 4. **Drawdown Expansion**: $Max\ Drawdown_{Candidate} > Max\ Drawdown_{Baseline} + 1.0\%$
-

11. Complete Repository Layout

```
argus-trader/
├── .github/
│   └── workflows/
│       ├── daily_batch.yml          # Scheduled 5:58 PM ET daily cron
│       └── backtest_regression_gate.yml # PR test gate & performance blocker
├── artifacts/
│   └── agents_data/                # Daily serialized agent outputs
│       ├── technicals/
│       ├── fundamentals/
│       ├── catalysts/
│       └── governance/
├── data/
│   ├── raw/
│   │   └── polygon/                # 5-year data in quarterly partitions (Parquet)
│   ├── pending_orders.json         # Staged CLI limit orders
│   ├── open_positions.json         # Active filled positions
│   ├── order_history.json          # Realized trades and outcomes
│   └── audit_ledger.json           # Retrospective classifications
├── docs/
│   ├── index.html                 # Live GitHub Pages dashboard
│   └── assets/                    # Styling and front-end scripts
├── scripts/
│   ├── fetch_historical_polygon.py # 5-year backfill script
│   ├── manage_orders.py           # Order registration CLI & daily reconciliation
│   ├── retrospective_audit.py      # Post-trade failure analysis engine
│   ├── publish_dashboard.py        # Jinja2 template rendering and git push
│   └── verify_regression_gate.py    # CI/CD PR evaluation script
├── src/
│   ├── tier0_macro/               # SPY, QQQ, VIX regime engine
│   ├── tier1_screener/            # Vectorized multi-archetype union
│   ├── tier2_agents/              # Technical, fundamental, catalyst leads
│   └── governance/                # Bull/Bear debate & Executive arbitration
├── templates/
│   └── dashboard.jinja            # Dashboard HTML template
├── tests/
│   ├── run_full_backtest.py        # 5-year backtesting simulation runner
│   └── test_extrema_zigzag.py      # Unit tests for pivot math
├── test_results/
│   └── benchmark_latest.json       # Canonical production baseline
├── requirements.txt               # Core dependencies
└── README.md                     # Repository overview and instructions
```

12. Open Pre-Implementation Decisions

The following operational parameter will be settled prior to initiating codebase construction:

- **LLM Candidate Cap & Batch Ingestion:** Determine whether Tier 1 passes the top 15 or all 30–35 candidates to Tier 2, and decide between standard synchronous API dispatch at 6:00 PM ET or queued asynchronous 24-hour Batch API processing.